

UniMentee Backend – Complete Implementation Roadmap (FastAPI + PostgreSQL + Supabase)

This document gives you the full staged plan for building the backend properly using layered architecture.

Stack: - FastAPI - PostgreSQL (Supabase hosted) - SQLAlchemy ORM - Alembic (migrations) - JWT Authentication - Bcrypt password hashing - Layered architecture (Router → Service → Repository → DB)

STAGE 0 — Project Setup

0.1 Create Project

```
mkdir unimentee-backend
cd unimentee-backend
python -m venv venv
venv\Scripts\activate # Windows
```

0.2 Install Dependencies

```
pip install fastapi uvicorn sqlalchemy pycpg2-binary python-dotenv alembic
passlib[bcrypt] python-jose pydantic[email]
```

Create requirements file:

```
pip freeze > requirements.txt
```

STAGE 1 — Core Infrastructure

Folder Structure

```
app/
  main.py
  config.py
  database.py
  core/
```

```
security.py
rbac.py
dependencies.py
exceptions.py
```

1.1 config.py

```
import os
from dotenv import load_dotenv

load_dotenv()

DATABASE_URL = os.getenv("DATABASE_URL")
JWT_SECRET = os.getenv("JWT_SECRET", "supersecret")
JWT_ALGORITHM = "HS256"
ACCESS_TOKEN_EXPIRE_MINUTES = 480
```

1.2 database.py

```
from sqlalchemy import create_engine
from sqlalchemy.orm import sessionmaker, declarative_base
from app.config import DATABASE_URL

engine = create_engine(DATABASE_URL, pool_pre_ping=True)

SessionLocal = sessionmaker(autocommit=False, autoflush=False, bind=engine)

Base = declarative_base()

def get_db():
    db = SessionLocal()
    try:
        yield db
    finally:
        db.close()
```

1.3 main.py

```
from fastapi import FastAPI

app = FastAPI(title="UniMentee ERP API")

@app.get("/")
def root():
    return {"status": "Backend running"}
```

Run server:

```
uvicorn app.main:app --reload
```

STAGE 2 — User + Authentication Module

Add folders:

```
models/
schemas/
services/
repositories/
routers/
```

2.1 models/users.py

```
from sqlalchemy import Column, BigInteger, String, Boolean, ForeignKey
from app.database import Base

class User(Base):
    __tablename__ = "users"

    user_id = Column(BigInteger, primary_key=True, index=True)
    university_id = Column(BigInteger, nullable=False)
    full_name = Column(String, nullable=False)
    email = Column(String, unique=True, nullable=False)
    hashed_password = Column(String, nullable=False)
    status = Column(String, default="ACTIVE")
```

2.2 schemas/auth.py

```
from pydantic import BaseModel, EmailStr

class LoginRequest(BaseModel):
    email: EmailStr
    password: str

class TokenResponse(BaseModel):
    access_token: str
    token_type: str = "bearer"
```

2.3 core/security.py

```
from passlib.context import CryptContext
from jose import jwt
from datetime import datetime, timedelta
from app.config import JWT_SECRET, JWT_ALGORITHM, ACCESS_TOKEN_EXPIRE_MINUTES

pwd_context = CryptContext(schemes=["bcrypt"], deprecated="auto")

def hash_password(password: str):
    return pwd_context.hash(password)

def verify_password(plain, hashed):
    return pwd_context.verify(plain, hashed)

def create_access_token(data: dict):
    to_encode = data.copy()
    expire = datetime.utcnow() + timedelta(minutes=ACCESS_TOKEN_EXPIRE_MINUTES)
    to_encode.update({"exp": expire})
    return jwt.encode(to_encode, JWT_SECRET, algorithm=JWT_ALGORITHM)
```

2.4 auth_service.py

```
from sqlalchemy.orm import Session
from app.repositories.user_repository import get_user_by_email
from app.core.security import verify_password, create_access_token

def authenticate_user(db: Session, email: str, password: str):
    user = get_user_by_email(db, email)
    if not user:
        return None
    if not verify_password(password, user.hashed_password):
        return None
    return user

def login_user(db: Session, email: str, password: str):
    user = authenticate_user(db, email, password)
    if not user:
        return None
    token = create_access_token({"sub": str(user.user_id), "university_id":
user.university_id})
    return token
```

2.5 auth router

```
from fastapi import APIRouter, Depends, HTTPException
from sqlalchemy.orm import Session
from app.database import get_db
from app.schemas.auth import LoginRequest, TokenResponse
from app.services.auth_service import login_user

router = APIRouter(prefix="/auth", tags=["Auth"])

@router.post("/login", response_model=TokenResponse)
def login(data: LoginRequest, db: Session = Depends(get_db)):
    token = login_user(db, data.email, data.password)
    if not token:
        raise HTTPException(status_code=401, detail="Invalid credentials")
    return {"access_token": token}
```

STAGE 3 — RBAC + Multi-Tenancy

core/rbac.py

```
from fastapi import Depends, HTTPException
from jose import jwt
from app.config import JWT_SECRET, JWT_ALGORITHM
from fastapi.security import HTTPBearer

security = HTTPBearer()

def get_current_user(token=Depends(security)):
    try:
        payload = jwt.decode(token.credentials, JWT_SECRET,
                               algorithms=[JWT_ALGORITHM])
        return payload
    except:
        raise HTTPException(status_code=401, detail="Invalid token")
```

All service methods must accept `university_id` and filter queries accordingly.

STAGE 4 — Academic Domain

Modules to implement next:

- subjects
- subject_offerings
- faculty allocations
- curriculum
- specializations

Each module must follow pattern:

model → repository → service → router

STAGE 5 — Enrollment Engine (Transactional)

Critical transaction example:

```
from sqlalchemy.orm import Session

def enroll_student(db: Session, student_id, offering_id):
    try:
        offering =
db.query(SubjectOffering).with_for_update().filter_by(id=offering_id).first()
        if offering.current_enrollment >= offering.max_enrollment:
            raise Exception("Capacity full")

        enrollment = StudentEnrollment(student_id=student_id,
offering_id=offering_id)
        db.add(enrollment)

        offering.current_enrollment += 1
        db.commit()
    except:
        db.rollback()
        raise
```

STAGE 6 — Mentor Module

Implement: - mentor_assignments - mentoring_sessions - section-level inheritance logic

Ensure partial unique constraint exists in DB.

STAGE 7 — Attendance + Auto Lock

Implement: - attendance_sessions - attendance_records - Auto-lock job (cron or background task)

STAGE 8 — Marks + SGPA Engine

When offering transitions to COMPLETED:

- Fetch all student marks
- Compute SGPA
- Insert into student_academic_progress
- Recompute CGPA

All inside transaction.

STAGE 9 — Portfolio + Storage Service

Create abstraction:

```
class StorageService:
    def generate_upload_url(self, file_key: str):
        pass
```

Never store file binary in DB.

STAGE 10 — Audit Logging

Create audit_service.py

```
def log_action(db, actor_id, action, entity, entity_id, old_value, new_value):
    audit = AuditLog(
        actor_user_id=actor_id,
        action=action,
        entity_type=entity,
        entity_id=entity_id,
        old_value=old_value,
        new_value=new_value,
    )
    db.add(audit)
```

Call this inside service layer only.

FINAL STAGE — Production Ready

- Add Alembic migrations
 - Add Docker
 - Add logging config
 - Add global exception handlers
 - Add pagination
 - Add index optimizations
-

Execution Order Summary

1. Core setup
 2. Auth
 3. RBAC
 4. Academic structure
 5. Enrollment engine
 6. Mentorship
 7. Attendance
 8. Marks + SGPA
 9. Portfolio
 10. Audit logging hardening
-

This is your full backend roadmap. Build module by module. Do not skip layers.

Next step: We begin implementing Stage 1 properly and cleanly.